

Implementation of Scientific Calculator Functions on Microcontroller using C

Krishna Patil, Nara Prajwal
Department of Electrical Engineering
IIT Hyderabad

Email: ee24btech11036@iith.ac.in, ee24btech11051@iith.ac.in

Abstract—This paper presents the implementation of scientific calculator functions such as trigonometric, logarithmic, exponential, power, factorial, and expression parsing using the C programming language on a microcontroller platform. Various numerical algorithms such as Runge–Kutta (RK4), series expansions, iterative methods, and parsing algorithms have been employed. Each function is discussed with its algorithmic basis followed by its C implementation.

Index Terms—Scientific Calculator, Microcontroller, Runge–Kutta Method, Quake III Algorithm, Shunting Yard Algorithm, C Programming

I. SOFTWARE IMPLEMENTATION

Instead of using “general expansion methods” (like Taylor/Maclaurin series with many terms), we reformulated each function as a differential equation. The algorithms used to calculate the values are RK4 (for ODEs), Newton–Raphson (for refinements like inverse sqrt).

A. Supported Functions

TABLE I
SUPPORTED FUNCTIONS AND OPERATIONS

Category	Functions / Operations
Trigonometric	$\sin(x)$, $\cos(x)$, $\tan(x)$
Inverse Trigonometric	$\arcsin(x)$, $\arccos(x)$, $\arctan(x)$
Exponential / Logarithmic	e^x , $\ln(x)$
Power / Root	x^y , $\frac{1}{\sqrt{x}}$
Factorial	$n!$
Basic Arithmetic	$+$, $-$, $\times$, $\div$
Constants	π , e
Input Symbols	Digits 0–9, decimal point, parentheses (,)

B. Runge–Kutta Method (RK4)

We employ the fourth-order Runge–Kutta (RK4) method [1], [3] to solve ordinary differential equations for trigonometric and exponential functions. This was chosen over alternative numerical integration techniques, keeping in mind the limited resources in a microcontroller based environment. A detailed analysis is given in Appendix B. The full derivation and step-by-step procedure for RK4 are provided in Appendix A.

C. Trigonometric Functions

1) *Sine Function*: The sine function [4] is computed by solving the second-order differential equation:

$$\frac{d^2y}{dx^2} + y = 0, \quad y(0) = 0, \quad y'(0) = 1 \quad (1)$$

using the fourth-order Runge–Kutta method.

System:

$$\frac{dy}{dx} = z, \quad \frac{dz}{dx} = -y, \quad y(0) = 0, \quad z(0) = 1 \quad (2)$$

The k_1, k_2, k_3, k_4 corresponds to the RK-4 terms of the equation $\frac{dy}{dx} = z$ and the l_1, l_2, l_3, l_4 corresponds to the RK-4 terms of the equation $\frac{dz}{dx} = -y$

RK4 step values:

$$k_1 = h z_n, \quad (3)$$

$$l_1 = -h y_n, \quad (4)$$

$$k_2 = h \left(z_n + \frac{l_1}{2} \right), \quad (5)$$

$$l_2 = -h \left(y_n + \frac{k_1}{2} \right), \quad (6)$$

$$k_3 = h \left(z_n + \frac{l_2}{2} \right), \quad (7)$$

$$l_3 = -h \left(y_n + \frac{k_2}{2} \right), \quad (8)$$

$$k_4 = h(z_n + l_3), \quad (9)$$

$$l_4 = -h(y_n + k_3), \quad (10)$$

$$y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4), \quad (11)$$

$$z_{n+1} = z_n + \frac{1}{6}(l_1 + 2l_2 + 2l_3 + l_4). \quad (12)$$

2) *Cosine Function*: The cosine function [4] is computed by $\sin(\frac{\pi}{2} - x)$.

3) *Tangent Function*: Tangent [4] is implemented as the ratio of sine and cosine:

D. Inverse Square Root

1) *Inverse Square Root*: The fast inverse square root algorithm (Quake III method) is used to efficiently compute $x^{-1/2}$. Detailed derivation and Newton–Raphson refinement are included in Appendix A.

E. Inverse Trigonometric Functions

1) *Arcsine*: The ODE for the arcsine function [4] is ODE:

$$\frac{dy}{dx} = \frac{1}{\sqrt{1-x^2}}, \quad y(0) = 0 \quad (13)$$

2) *Arccosine*: Use the identity [4]

$$\arccos(x) = \frac{\pi}{2} - \arcsin(x) \quad (14)$$

3) *Arctangent*: The ODE for the arctan function [4] is ODE:

$$\frac{dy}{dx} = \frac{1}{1+x^2}, \quad y(0) = 0 \quad (15)$$

F. Logarithmic Functions

The natural logarithm can be obtained by solving the differential equation [4]

$$\frac{dy}{dx} = \frac{1}{x}, \quad y(1) = 0 \quad (16)$$

which has the exact solution $y(x) = \ln(x)$. Using the RK4 method, we can approximate $\ln(x)$ by integrating this ODE from $x = 1$ to the target value.

G. Exponential Functions

1) *Power Function*: The power function $y = x^w$ [4] satisfies

$$\frac{dy}{dx} = \frac{w}{x}y, \quad y(1) = 1 \quad (17)$$

H. Factorial Function

The algorithm is a simple recursive function in C

$$y_n = n \cdot y_{n-1} \quad (18)$$

[2]

Algorithm 1 Recursive Factorial Function

```

1: function FACTORIALRECURSIVE(x)
2:   if  $x \leq 1$  then
3:     return 1
4:   else
5:     return  $x \times \text{FactorialRecursive}(x - 1)$ 
6:   end if
7: end function

```

I. Expression Parsing

The Shunting Yard algorithm is employed for converting infix expressions into postfix form for evaluation.

J. Overview

The **Shunting Yard Algorithm**, proposed by Edsger Dijkstra, is a method for converting mathematical expressions in infix notation into postfix notation (Reverse Polish Notation, RPN). It relies on two structures:

- A **stack** for operators/functions
- An **output queue** for the final postfix expression

K. Algorithm Steps

- 1) Initialize an empty stack and output queue.
- 2) For each token:
 - a) Numbers $\rightarrow$ add to output.
 - b) Functions $\rightarrow$ push to stack.
 - c) Operators $\rightarrow$ pop higher/equal precedence operators from stack to output (respecting associativity), then push current operator.
 - d) "(" $\rightarrow$ push to stack.
 - e) ")" $\rightarrow$ pop to output until matching "(" is found, discard brackets.
- 3) After processing, pop remaining operators to output.

Algorithm 2 Shunting Yard Algorithm (Infix to Postfix)

```

1: function SHUNTINGYARD(expr)
2:   Initialize empty stack operators
3:   Initialize empty string output
4:   for each character c in expr do
5:     if c is a number or '.' then
6:       Append c to output until number ends
7:       Append space to output
8:     else if c is a function then
9:       Push c onto operators
10:    else if c = ( then
11:      Push c onto operators
12:      Increment unmatched_brackets
13:    else if c = ) then
14:      while Top of operators  $\neq$  ( do
15:        Pop from operators and append to output
16:      end while
17:      Pop ')' from operators
18:      Decrement unmatched_brackets
19:    if Top of operators is a function then
20:      Pop function and append to output
21:    end if
22:    else if c is an operator in {+, -, *, /, !} then
23:      while operators not empty and precedence(Top of operators)  $\geq$  precedence(c) do
24:        Pop from operators and append to output
25:      end while
26:      Push c onto operators
27:    end if
28:  end for
29:  while operators not empty do
30:    Pop from operators and append to output
31:  end while
32:  return output
33: end function

```

L. Operator Precedence

M. Example

For the expression $3 + 4 * 2 / (1 - 5)$, the Shunting Yard Algorithm yields the postfix form:

$$3 \ 4 \ 2 \ * \ 1 \ 5 \ - \ / \ + \quad (19)$$

TABLE II
RK4 UPDATE EQUATIONS FOR SINGLE-VARIABLE FUNCTIONS

Function	k_1	k_2	k_3	k_4	y_{n+1}
Arcsine $\arcsin(x)$	$\frac{h}{\sqrt{1-x_n^2}}$	$\frac{h}{\sqrt{1-(x_n+\frac{h}{2})^2}}$	$\frac{h}{\sqrt{1-(x_n+\frac{h}{2})^2}}$	$\frac{h}{\sqrt{1-(x_n+h)^2}}$	$y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$
Arctangent $\arctan(x)$	$\frac{h}{1+x_n^2}$	$\frac{h}{1+(x_n+\frac{h}{2})^2}$	$\frac{h}{1+(x_n+\frac{h}{2})^2}$	$\frac{h}{1+(x_n+h)^2}$	$y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$
Natural Log $\ln(x)$	$\frac{h}{x_n}$	$\frac{h}{x_n+\frac{h}{2}}$	$\frac{h}{x_n+\frac{h}{2}}$	$\frac{h}{x_n+h}$	$y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$
Power $y = x^w$	$h \cdot \frac{w}{x_n} y_n$	$h \cdot \frac{w}{x_n+\frac{h}{2}} \left(y_n + \frac{k_1}{2}\right)$	$h \cdot \frac{w}{x_n+\frac{h}{2}} \left(y_n + \frac{k_2}{2}\right)$	$h \cdot \frac{w}{x_n+h} (y_n + k_3)$	$y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$

TABLE III
OPERATOR PRECEDENCE AND ASSOCIATIVITY

Operator	Precedence	Associativity
$\vee$ (power)	4	Right
$*, /$	3	Left
$+, -$	2	Left

Algorithmic Steps :

TABLE IV
SHUNTING YARD EXAMPLE FOR $3 + 4 * 2 / (1 - 5)$

Token	Action	Stack	Output
3	Add to output		3
+	Push to stack	+	3
4	Add to output	+	3 4
*	Push to stack	+ *	3 4
2	Add to output	+ *	3 4 2
/	Pop *, push /	+ /	3 4 2 *
(	Push to stack	+ / (	3 4 2 *
1	Add to output	+ / (	3 4 2 * 1
-	Push to stack	+ / (-	3 4 2 * 1
5	Add to output	+ / (-	3 4 2 * 1 5
)	Pop until (	+ /	3 4 2 * 1 5 -
End	Pop stack		3 4 2 * 1 5 - / +

N. Reverse Polish Notation (RPN)

Reverse Polish Notation, also known as postfix notation, is a way of writing mathematical expressions in which operators follow their operands. Unlike infix notation (e.g., $3 + 4$), RPN eliminates the need for parentheses by unambiguously encoding operator precedence.

- Example (infix): $3 + 4$
- Equivalent RPN: $3\ 4\ +$

This simplicity makes RPN especially suitable for evaluation using a stack-based algorithm.

O. Evaluating RPN

Evaluation of RPN uses a stack:

- 1) Scan tokens left to right.
- 2) Push numbers on the stack.
- 3) For operators, pop required operands, apply operation, push result.

For the above example, evaluating the postfix expression gives the result 1 as expected .

P. Complexity and Applications

The algorithm runs in $O(n)$ time, where n is the number of tokens. Combined with RPN evaluation, it provides an efficient method for handling expressions in calculators, compilers, and symbolic computation systems.

Q. Function Handling

The Shunting Yard Algorithm can also be used to implement functions such as $\sin(x)$, $\cos(x)$, and others. Function handling follows these rules:

- 1) When a function token is encountered, it is pushed onto the operator stack.
- 2) Arguments are processed as sub-expressions normally.
- 3) When the closing parenthesis “)” is reached:
 - Operators are popped from the stack to the output queue until the matching “(” is found.
 - The function token itself is then moved from the stack to the output queue.

As a result, function calls are correctly represented in postfix form. For example:

$$\sin(x) \rightarrow x \sin, \quad (20)$$

R. Conclusion

The Shunting Yard Algorithm, together with RPN evaluation, ensures efficient parsing and evaluation of mathematical expressions. It respects operator precedence, associativity, and bracket handling, making it a cornerstone in expression processing across computing applications.

S. Code Repository

The complete C implementations of the algorithms discussed in this paper are available at: <https://github.com/gadepall/calculator/tree/main/codes>. Instructions for installation of a software based prototype for testing the algorithms are given in Appendix C.

```

1 codes/
2 |-- ShuntingYard.c
3 |-- inv_sq_root.c
4 |-- inv_trig_func.c

```

```

5 |-- log.c
6 |-- power_func.c
7 |-- trig_func.c

```

Listing 1. Repository Structure in codes/

II. HARDWARE IMPLEMENTATION

A. Hardware Required

TABLE V
MATERIALS REQUIRED FOR THE SCIENTIFIC CALCULATOR

Quantity	Component
25	Pushbuttons
1	LCD 16×2
1	Arduino Uno
–	Jumper wires
1	Potentiometer

- A button matrix for user input.
- An Arduino Uno microcontroller to process inputs and execute calculations.
- A 16×2 LCD connected to Arduino for showing results.
- Connections as shown in Fig. 2.

B. Circuit Connections

TABLE VI
ARDUINO TO LCD PIN CONNECTIONS

Arduino Pin	LCD Pin	LCD Label	Description
GND	1	GND	Ground
5V	2	Vcc	Power Supply
GND	3	Vee	Contrast Control
D2	4	RS	Register Select
GND	5	R/W	Read/Write (fixed to Write)
D3	6	EN	Enable
D4	11	DB4	Data Bus (4-bit mode)
D5	12	DB5	Data Bus (4-bit mode)
D6	13	DB6	Data Bus (4-bit mode)
D7	14	DB7	Data Bus (4-bit mode)
5V	15	LED+	Backlight Anode
GND	16	LED-	Backlight Cathode

C. Button Matrix

The button matrix is a grid of push-buttons arranged in rows and columns. It allows multiple buttons to be connected to the microcontroller using fewer pins.

Working Principle:

- The Arduino scans the matrix by activating each row (setting it LOW) one at a time while reading the columns.
- When a button is pressed, it completes the circuit.
- By identifying the active row and column, the Arduino determines which button was pressed.
- Example: If second row is pressed when first column is active, the first column's second button is pressed.

TABLE VII
KEY ASSIGNMENTS IN MODE 1 WITH ARDUINO PIN MAPPING

	Col 1	Col 2	Col 3	Col 4	Col 5
Row 1	0	1	2	3	4
Row 2	5	6	7	8	9
Row 3	+	-	×	÷	sin(
Row 4	cos(	tan(	e(	ln(	Clear
Row 5	Backspace	.	=	Mode shift	π

TABLE VIII
KEY ASSIGNMENTS IN MODE 2 WITH ARDUINO PIN MAPPING

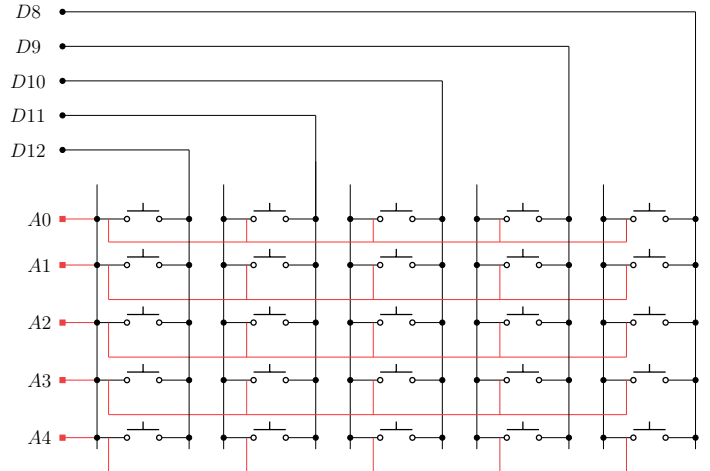
	Col 1	Col 2	Col 3	Col 4	Col 5
Row 1	0	1	2	3	4
Row 2	5	6	7	8	9
Row 3	(	)	x^y	fact(	arcsin(
Row 4	arccos(	arctan(	mod	$\log_{10}($	Clear
Row 5	Backspace	.	=	Mode shift	π

This reduces the number of pins required to implement a calculator with 25 buttons.

TABLE IX
KEYPAD ROW/COLUMN CONNECTIONS TO ARDUINO PINS

Keypad Line	Arduino Pin
Row 1	D8
Row 2	D9
Row 3	D10
Row 4	D11
Row 5	A3
Col 1	D12
Col 2	D13
Col 3	A0
Col 4	A1
Col 5	A2

Fig. 1. Button Matrix



D. Schematic Circuit

The schematic for circuit connections is shown below,

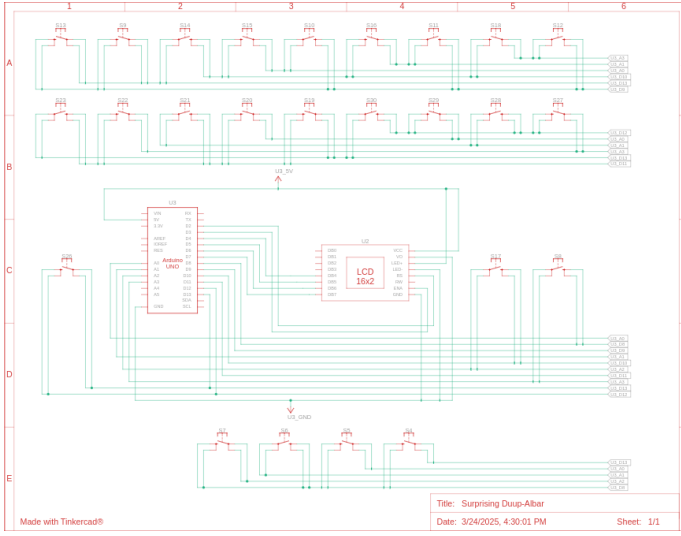


Fig. 2. Schematic of Circuit.

E. Hardware Circuit

The hardware circuit of calculator,

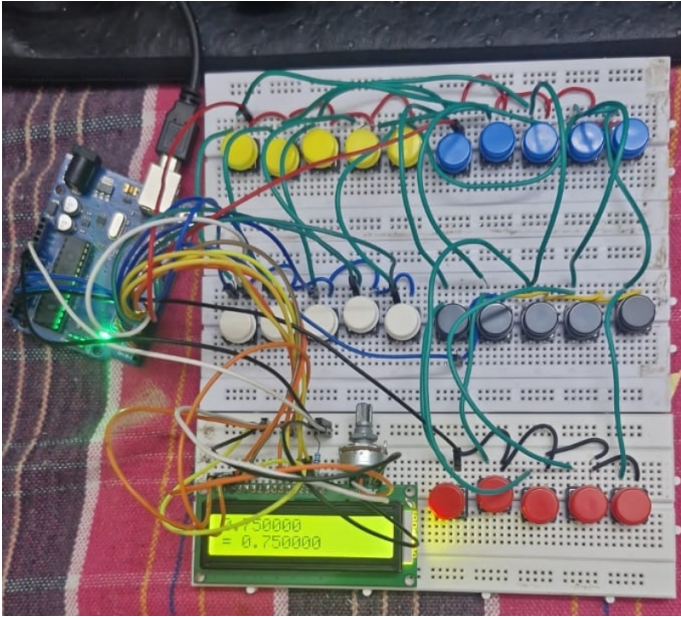


Fig. 3. circuit

APPENDIX A RUNGE-KUTTA METHOD (RK4)

The RK4 method is a numerical technique to solve

$$\frac{dy}{dx} = f(x, y), \quad y(x_0) = y_0 \quad (21)$$

with step size h . The next value is calculated as:

$$y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (22)$$

where

$$k_1 = hf(x_n, y_n), \quad (23)$$

$$k_2 = hf(x_n + h/2, y_n + k_1/2), \quad (24)$$

$$k_3 = hf(x_n + h/2, y_n + k_2/2), \quad (25)$$

$$k_4 = hf(x_n + h, y_n + k_3). \quad (26)$$

Quake III Algorithm: The method obtains an initial approximation by manipulating the IEEE 754 floating-point representation of x , then refines it using Newton-Raphson iterations.

- 1) Bit manipulation with a “magic constant” ($0x5f3759df$) produces an initial guess y_0 .
- 2) A single Newton-Raphson iteration dramatically improves accuracy.
- 3) An optional second iteration yields nearly full precision.

Newton-Raphson Refinement: We want to approximate

$$y = x^{-\frac{1}{2}}. \quad (27)$$

Define

$$f(y) = \frac{1}{y^2} - x, \quad f'(y) = -\frac{2}{y^3}. \quad (28)$$

Applying Newton-Raphson,

$$y_{k+1} = y_k - \frac{f(y_k)}{f'(y_k)} \quad (29)$$

$$= y_k - \frac{\frac{1}{y_k^2} - x}{-\frac{2}{y_k^3}} \quad (30)$$

$$= y_k + \frac{y_k}{2} (1 - xy_k^2) \quad (31)$$

$$= y_k \left(\frac{3}{2} - \frac{1}{2}xy_k^2 \right). \quad (32)$$

Thus one Newton iteration is simply

$$y \leftarrow y (1.5 - 0.5xy^2). \quad (33)$$

APPENDIX B COMPARATIVE ANALYSIS OF NUMERICAL METHODS

Three criteria were considered in the choosing of RK4 as the algorithm for a microcontroller base:

- Computing complexity
- Accuracy for the chosen precision level
- Memory usage

A. Rejected Alternative Algorithms

1) *Euler's Method*: The Euler method is the simplest numerical solver. The basic idea is to approximate the curve by its tangent line, meaning it is a first order solver.

- **Forward Euler (Explicit)** The slope at the current point is used as the approximation for the value at the end of the step.

$$y_{n+1} = y_n + h \cdot f(x_n, y_n) \quad (34)$$

- **Backward Euler (Implicit)** The slope at the end of the step is used as the approximation. y_{n+1} is present on both sides of the equation, so another algorithm is needed to separate it at each step.

$$y_{n+1} = y_n + h \cdot f(x_{n+1}, y_{n+1}) \quad (35)$$

- **Drawback:** For a step size of h , the error is $O(h)$. Thus to reduce error, we would have to increase step size drastically, leading to lots of computations.

2) *Trapezoidal Rule*: This method approximates the integral by dividing the area under the curve into trapezoids instead of rectangles.

$$y_{n+1} = y_n + \frac{h}{2} \cdot (f(x_n, y_n) + f(x_{n+1}, y_{n+1})) \quad (36)$$

- **Drawbacks:** It is second order, meaning error of $O(h^2)$. It is also implicit, meaning each step would require another iterative solving mechanism such as Newton-Rhapson.

3) *Series Expansion Methods*: Taylor/Maclaurin series can also be used for this purpose.

- **Drawbacks:** Precision cannot be achieved consistently as the number of terms needed vary with function and also with input value. As the input increases, convergence is slow, and number of terms needed to be stored in memory increases.

4) *Second-Order Runge-Kutta Methods (RK2)*: A step up in accuracy from Euler's method are the second-order Runge-Kutta (RK2) methods. These methods use a preliminary slope calculation to make a more accurate step.

- **Heun's Method**: This method averages the slope at the beginning and end of the interval (using an Euler step as a predictor):

$$\begin{aligned} \tilde{y}_{n+1} &= y_n + h \cdot f(x_n, y_n) \\ y_{n+1} &= y_n + \frac{h}{2} (f(x_n, y_n) + f(x_{n+1}, \tilde{y}_{n+1})) \end{aligned}$$

- **Drawback:** Both methods are second-order, with global error proportional to h^2 (denoted $O(h^2)$). While this is a significant improvement over Euler's method, it is still much less accurate than the fourth-order RK4 method ($O(h^4)$). Given that RK4 only requires twice the number of function evaluations per step as these RK2 methods (four vs. two), the substantial gain in accuracy is a highly favorable trade-off for a scientific calculator application.

B. Justification for RK4 Selection

The fixed-step RK4 method was determined to be the optimal choice for this specific hardware implementation based on the following analysis (summarized in Table X). The code for this analysis can be found at: <https://github.com/gadepall/calculator/tree/main/analysis>:

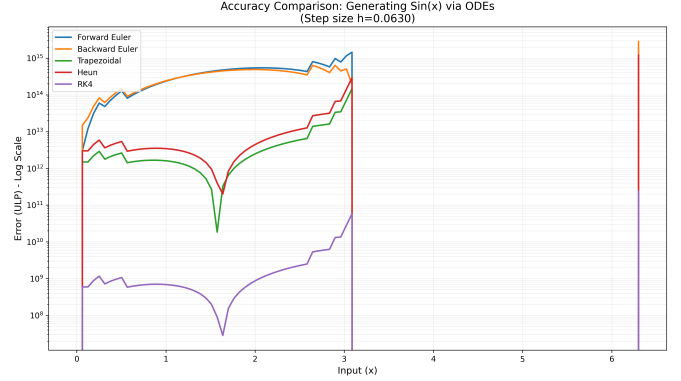


Fig. 4. Comparison of the accuracy of the algorithms

Here, each algorithm was compared against the gold-standard MPFR library up to 50 decimal places. Accuracy is represented using **Units in Last Place (ULP)** measure.

As per the results in Fig.4, RK4 demonstrates a clear dominance, with both maximum and average errors being many orders of magnitude less than the other algorithms. The Euler methods demonstrate an increasing drift from the true value with increase in input value, while the trapezoidal method, while stable in its error variation, featured much more absolute error than RK4. As expected, RK2 showed similar error variation as RK4 but much greater in magnitude.

TABLE X
ACCURACY COMPARISON FOR $\sin(x)$ GENERATION ($h \approx 0.06$)

Algorithm	Max Error (ULP)	RMS Error	Stability
RK4	2.38×10^{11}	3.44×10^{-7}	High
Trapezoidal	6.00×10^{14}	8.77×10^{-4}	High
Heun	1.20×10^{15}	1.74×10^{-3}	Moderate
Forward Euler	1.86×10^{15}	8.56×10^{-2}	Unstable
Backward Euler	2.84×10^{15}	7.27×10^{-2}	Unstable

APPENDIX C SOFTWARE SETUP

A. Termux

1) On your android device, follow the instructions in

```
1 https://github.com/gadepall/fwc-1
```

to setup and install Debian on Termux.

B. Platfomio

1) Install Packages

```
1 apt install avra avrdude gcc-avr
avr-libc
```

2) Follow the instructions in

```
1 https://docs.platformio.org/en/stable/
  core/installation/methods/installer
  -script.html#super-quick-macos-
  linux
```

to install platformio.

3) Execute the following on debian

```
1 cd ide/piosetup/codes
2 pio run
```

See Fig. 5 for the Arduino pin diagram.

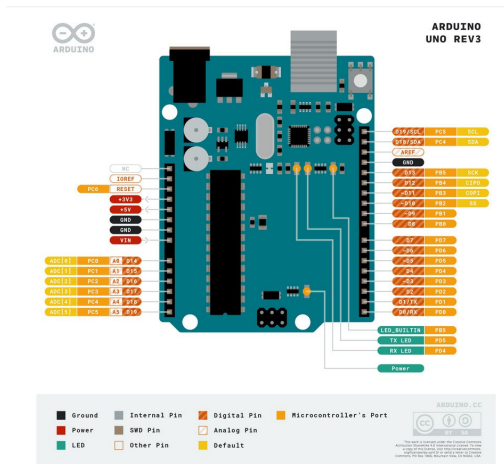


Fig. 5. Arduino Pinout

C. Arduino Droid

- 1) Install ArduinoDroid from apkpure
- 2) Open ArduinoDroid and grant all permissions
- 3) Connect the Arduino to your phone via USB-OTG
- 4) For flashing the bin files, in ArduinoDroid,

```
1 Actions->Upload->Upload Precompiled
```

then go to your working directory and select

```
1 .pio/build/uno/firmware.hex
```

for uploading hex file to the Arduino Uno

- 5) The LED beside pin 13 will start blinking

D. App Upload

- 1) Download the app APK file and install it from your File Manager.

```
1
```

REFERENCES

- [1] B. S. Grewal, *Higher Engineering Mathematics*, 43rd ed. New Delhi, India: Khanna Publishers, 2014.
- [2] G. V. V. Sharma, *C Programming in Middle School*
- [3] E. Kreyszig, *Advanced Engineering Mathematics*, 10th ed. Hoboken, NJ, USA: John Wiley & Sons, 2011.
- [4] National Council of Educational Research and Training (NCERT), *Mathematics: Textbook for Class XII*, Part 1 and 2, New Delhi, India: NCERT, 2006.